

ASSESSMENT

Section A

S1: Q1:

Which development methodology would you recommend for this project, and why? Identify two specific risks the team would face if they chose the wrong methodology for this context.

- **Recommended Methodology: Agile**
- **Reason:**

Agile is the best choice because the project has frequently changing requirements and the client wants to review working features every two weeks. Agile delivers software in short iterations (sprints), allowing regular client feedback and quick changes based on new requirements.
- **Two Risks of Choosing the Wrong Methodology (e.g., Waterfall):**
- **Difficulty handling changing requirements:** Waterfall requires requirements to be fixed at the beginning, so changes later can cause delays, increased costs, and rework.
- **Late client feedback:** The client sees the product only near the end of development, increasing the

risk that the final product does not meet the client's expectations.

- **Conclusion:**

Agile is more suitable because it supports continuous feedback, frequent releases, and flexibility, making it ideal for projects with changing requirements.

S2: Q2:

Describe the Git workflow your team should adopt to prevent this from happening again. Name at least two Git commands your team should use as part of this workflow and explain the role of each command.

Answer:

Recommended Git Workflow:

The team should use a feature branch workflow instead of pushing code directly to the main branch. Each team member should create a separate branch for their work, test the changes, and then submit a Pull Request (PR) for review. Only reviewed and approved code should be merged into the main branch.

Git Commands and Their Roles

1. git checkout -b feature-branch

- Creates a new branch and switches to it.
- Allows developers to work on new features without affecting the main branch.

2. git push origin feature-branch

- Uploads the feature branch to the remote repository.
- Enables teammates to review the code before it is merged.

3. git merge feature-branch

- Merges the approved feature branch into the main branch.
- Ensures only tested and reviewed code becomes part of the project.

Conclusion

By using feature branches, pull requests, and code reviews, the team can prevent incomplete or broken code from being pushed directly to the main branch, keeping the project stable for everyone.

S3: Q3:

List the appropriate HTML input types you would use for each of the five fields. Explain how you

would use HTML5 built-in validation attributes to enforce the two constraints mentioned, and why these attributes alone are not sufficient for production-level validation:

Field	HTML Input Type
Applicant Name	type="text"
Email Address	type="email"
Phone Number	type="tel"
Preferred Domain	<select> (Options: Web, Data, Mobile, AI)
Resume Upload	type="file"

S4: Q4:

Explain how Bootstrap's grid class system works to produce this responsive layout. Write the Bootstrap class combination you would apply to the article column div, and explain what each class in the combination controls.

Answer:

How Bootstrap's Grid System Works

Bootstrap uses a 12-column responsive grid system. By applying different grid classes for different screen sizes, the layout automatically changes without writing custom CSS media queries.

For this scenario:

- On mobile devices, each article takes the full width (12 columns), so articles are displayed in one column.
- On desktop screens, each article takes 4 columns out of 12, so 3 articles fit in one row ($4 \times 3 = 12$).

Bootstrap Class Combination

```
<div class="col-12 col-md-4">  
  <!-- Article Content -->  
</div>
```

Explanation of Each Class

- col-12
 - Makes the column span all 12 grid columns.
 - Used on extra-small and small screens (mobile).
 - Displays one article per row.

Example

```
<div class="container">  
  <div class="row">  
    <div class="col-12 col-md-4">Article 1</div>  
    <div class="col-12 col-md-4">Article 2</div>  
    <div class="col-12 col-md-4">Article 3</div>  
  </div>  
</div>
```

Conclusion

Using the Bootstrap classes col-12 col-md-4 creates a responsive layout:

- Mobile: 1 column (each article takes full width).
- Desktop: 3 columns (each article takes one-third of the row).

This approach uses Bootstrap's built-in responsive grid system and does not require any custom media query CSS.

S5: Q5:

Explain why an array is the correct data structure for this task compared to using 30 separate variables. Describe the two-step logic your program would follow: first to compute the average, then to identify the above-average days — and explain why both steps cannot be done in a single loop.

Answer:

Why an Array is the Correct Data Structure

An array is the best choice because it stores multiple values of the same data type under a single variable name. Instead of creating 30 separate variables (day1, day2, ..., day30), an array allows easy storage, access, and processing of all rainfall readings using loops.

Advantages of an array:

- Stores all 30 rainfall values in one variable.
- Reduces code repetition.
- Makes it easy to use loops for calculations.
- Simplifies maintenance and future changes.

Two-Step Program Logic

Step 1: Calculate the Average

- Store the rainfall values in an array.

- Use a loop to add all 30 readings.
- Calculate the average:
- $\text{average} = \text{total} / 30;$

Step 2: Find Above-Average Days

- Use another loop to compare each day's rainfall with the calculated average.
- If a day's rainfall is greater than the average, display that day's number and rainfall amount.

Why Both Steps Cannot Be Done in One Loop

The average cannot be known until all 30 rainfall values have been added together. During the first loop, the program is still calculating the total, so the final average is not yet available. Since the comparison requires the final average, a second loop is necessary to identify the days with rainfall above the average.

Conclusion

Using an array makes storing and processing the 30 rainfall readings efficient. The program first calculates the monthly average and then uses a second loop to find the days with above-average rainfall, because the average must be calculated before any comparisons can be made.

S6: Q6:

Explain why the program crashes on an empty string input and describe the check your pointer-based solution must perform before dereferencing the pointer. How does traversing a string using a pointer differ from traversing it using array index notation, and which approach makes the empty-input bug easier to catch?

Answer:

Why the Program Crashes

When the user presses Enter without typing a name, the string is empty (""). If the pointer is NULL or the program dereferences the pointer without checking whether it points to a valid character, it may access invalid memory, causing a segmentation fault (crash).

Check Before Dereferencing the Pointer

Before accessing (*ptr) the value pointed to by the pointer, the program should check:

- The pointer is not NULL.
- The current character is not the null terminator ('\0').

Example:

```
if (ptr != NULL && *ptr != '\0')
```

```
{  
    // Safe to access the character  
}
```

Pointer Traversal vs Array Index Traversal

Pointer Traversal

```
char *ptr = str;  
while (*ptr != '\0')  
{  
    printf ("%c", *ptr);  
    ptr++;  
}
```

- Uses a pointer that moves from one character to the next.
- Accesses characters using pointer dereferencing (*ptr).

Array Index Traversal

```
for (int i = 0; str[i] != '\0'; i++)  
{  
    printf ("%c", str[i]);  
}
```

- Uses an index (i) to access each character.

- Accesses characters using array notation (`str[i]`).

Which Approach Makes the Bug Easier to Catch?

Array index traversal is generally easier to understand and debug because it clearly checks `str[i] != '\0'` before accessing characters. Pointer traversal is efficient but requires extra care to ensure the pointer is valid before dereferencing it.

Conclusion

The crash occurs because the program dereferences a pointer without ensuring it points to a valid character. A pointer-based solution should check that the pointer is not NULL and that `*ptr != '\0'` before dereferencing. While both methods work, array index traversal usually makes empty-string handling easier to identify and debug.

